

pwn.college: File Struct

Introduction to stdio streams

Robert Wasinger
Arizona State University

What is a File Descriptor

What happens when we “open” a file?

```
int fd = open("/home/hacker/class_notes.txt", O_RDWR)
```

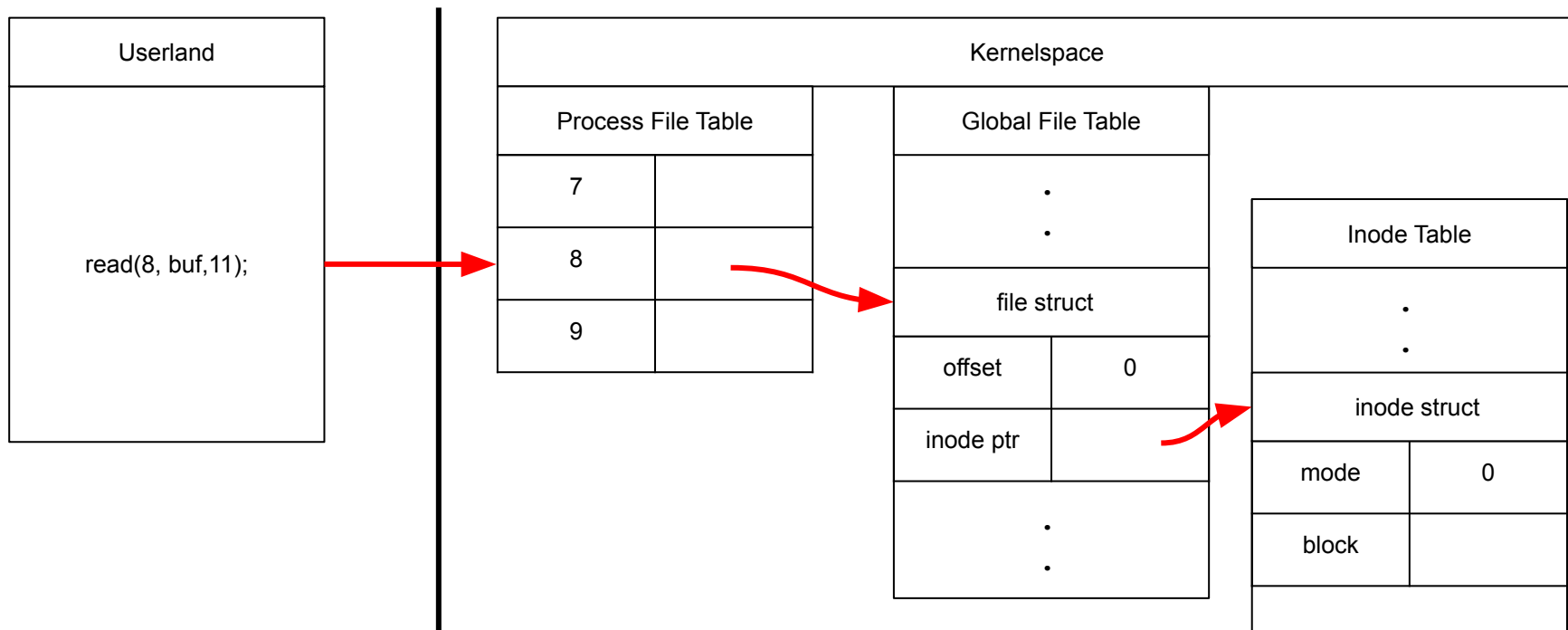
```
printf("%d\n", fd)
```

```
,
```

```
./a.out
```

```
3
```

File Descriptor Translation



File Descriptor Translation

This is a non-trivial amount of work that occurs every read/write!

Can we do better?

libc - stdio

libc has a class of functions that **CAN** be more performant! (DEMO!)

stdio streams use functions such as:

- `FILE *fopen(const char *pathname, const char *mode);`
- `size_t fread(void *ptr, size_t size, size_t nmemb, FILE *stream);`
- `size_t fwrite(const void *ptr, size_t size, size_t nmemb, FILE *stream);`
- many more!

How do they work?

FILE struct

These functions work on a **FILE** struct instead of a file descriptor

The **FILE** struct contains pointers used for read/write buffers!

This reduced the number of context switches, increasing I/O performance

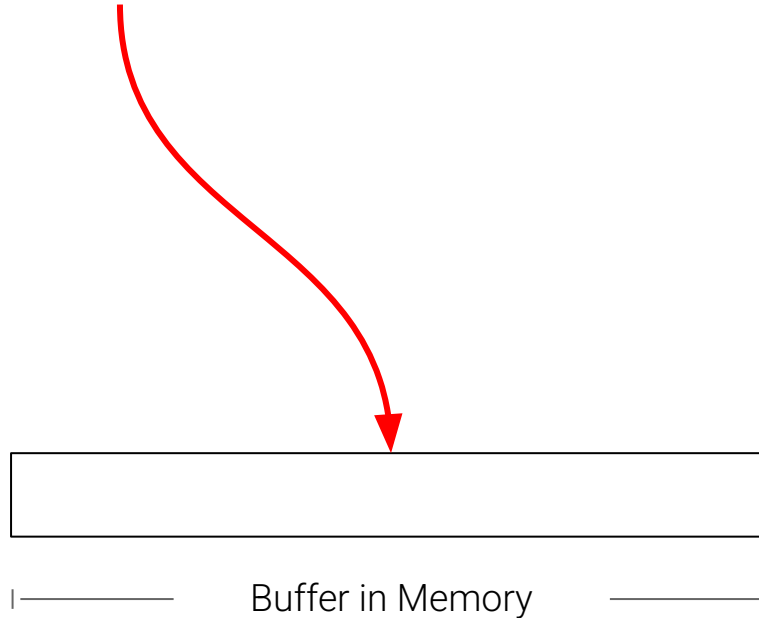
FILE struct - Buffer Pointers

```
int _flags;           /* High-order word is _IO_MAGIC; rest is flags. */
char *_IO_read_ptr;   /* Current read pointer */
char *_IO_read_end;   /* End of get area. */
char *_IO_read_base;  /* Start of putback+get area. */
char *_IO_write_base; /* Start of put area. */
char *_IO_write_ptr;   /* Current put pointer. */
char *_IO_write_end;   /* End of put area. */
char *_IO_buf_base;    /* Start of reserve area. */
char *_IO_buf_end;     /* End of reserve area. */
```

https://elixir.bootlin.com/glibc/glibc-2.31/source/libio/bits/types/struct_FILE.h#L49

FILE struct - Reading from File to Memory

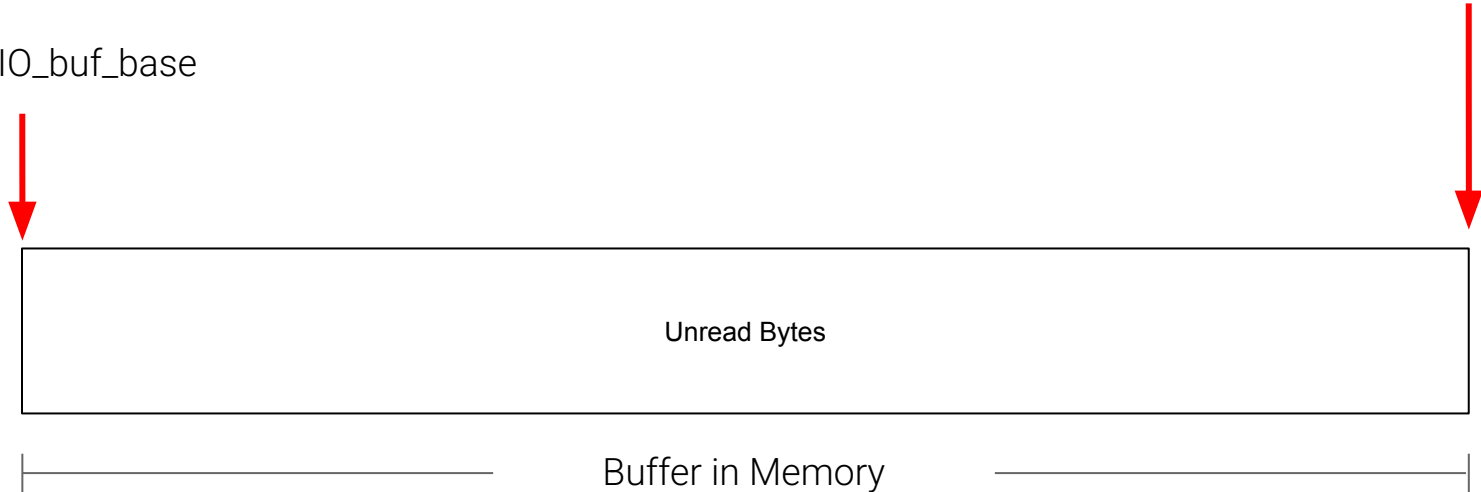
When we call `fread(buf, 1, 10, fp)` It checks a buffer, not the file!



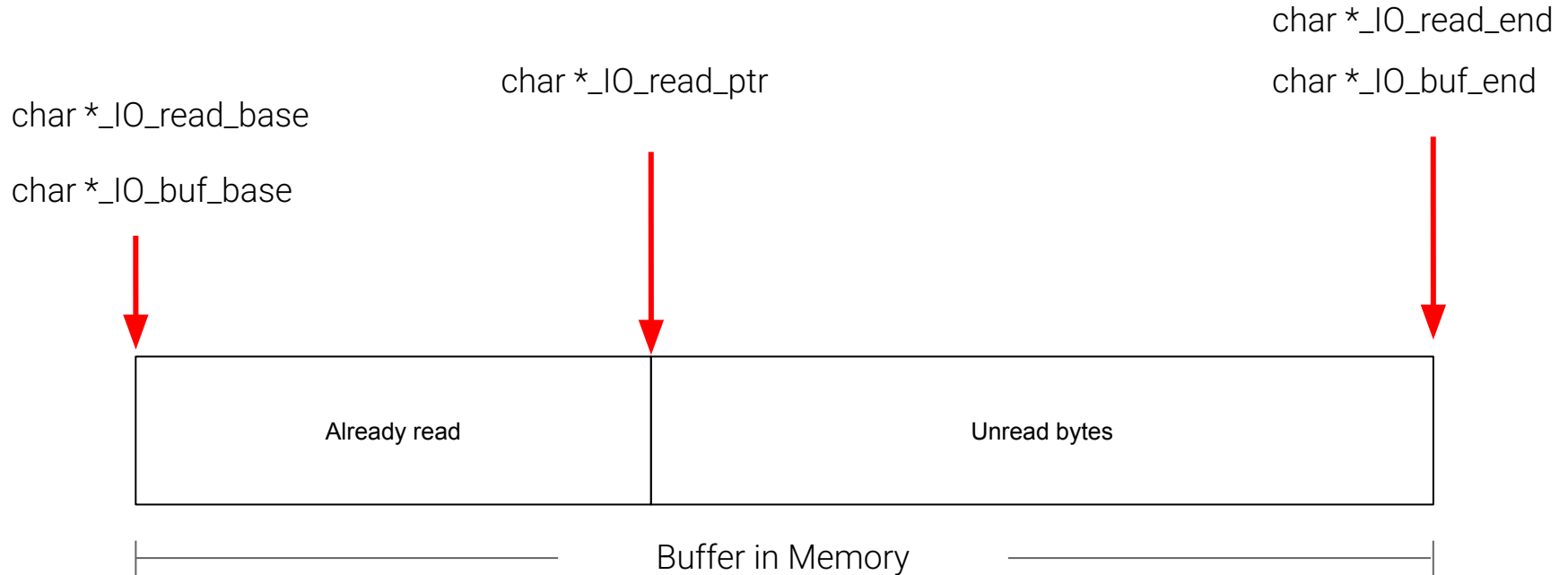
FILE struct - Reading from File to Memory

char *_IO_read_ptr
char *_IO_read_base
char *_IO_buf_base

char *_IO_read_end
char *_IO_buf_end



FILE struct - Reading from File to Memory



FILE struct - Reading from File to Memory

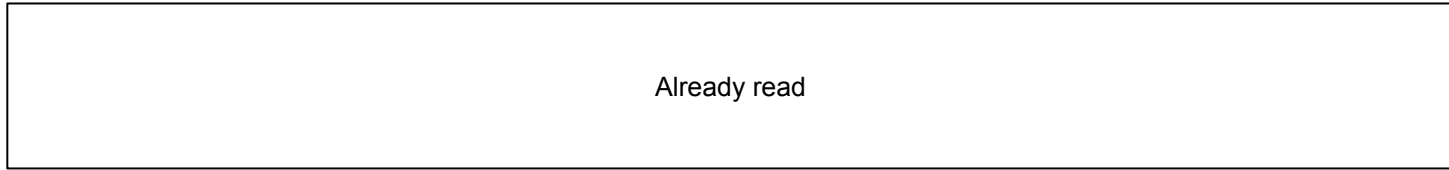
char *_IO_read_base

char *_IO_buf_base

char *_IO_read_ptr

char *_IO_read_end

char *_IO_buf_end

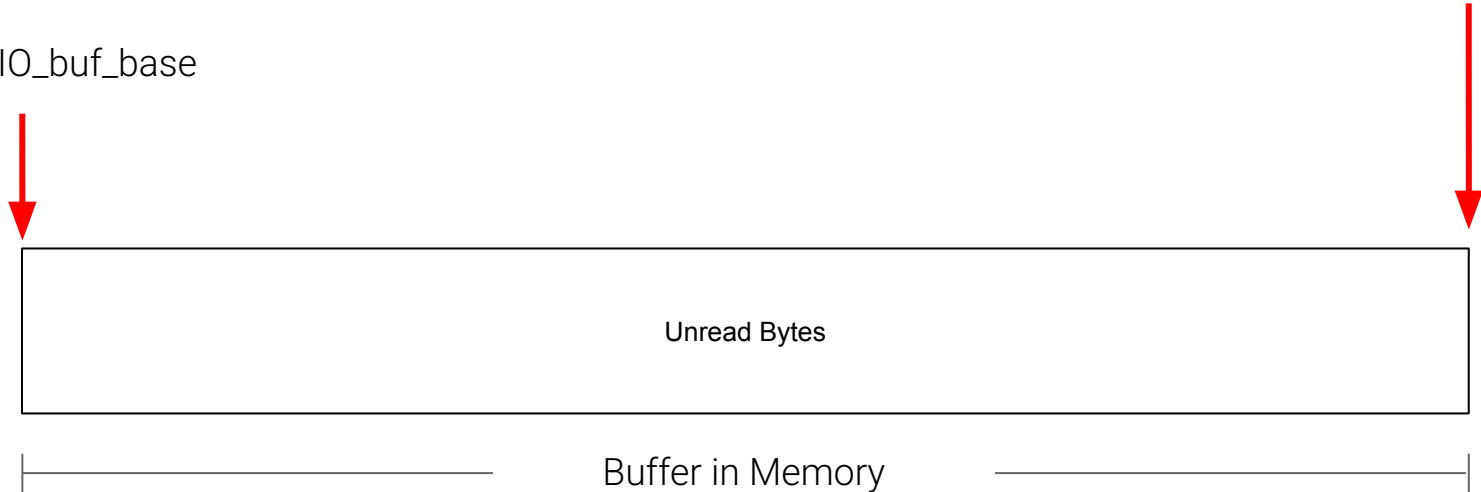


Buffer in Memory

FILE struct - Reading from File to Memory

char *_IO_read_ptr
char *_IO_read_base
char *_IO_buf_base

char *_IO_read_end
char *_IO_buf_end



FILE struct - Reading from File to Memory

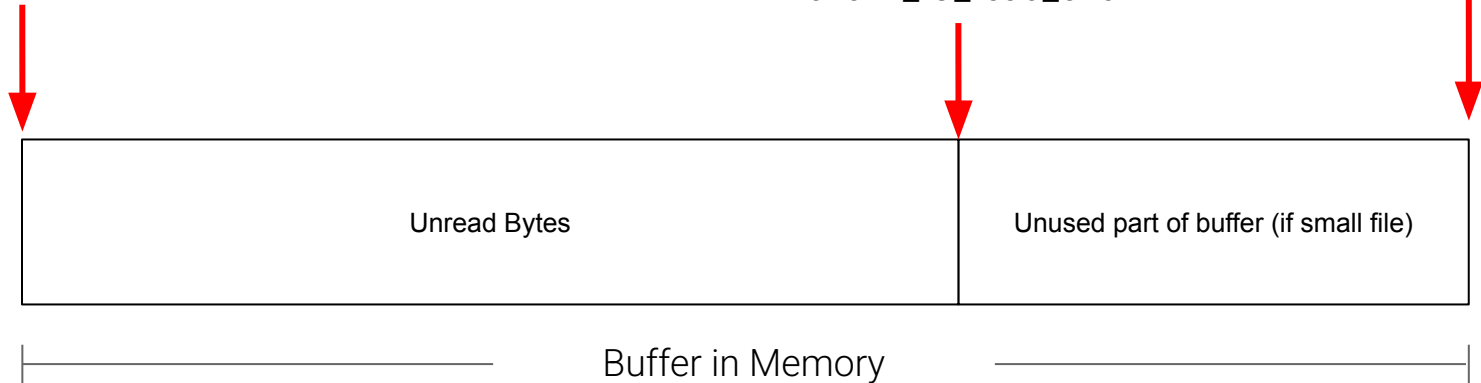
`char *_IO_read_ptr`

`char *_IO_read_base`

`char *_IO_buf_base`

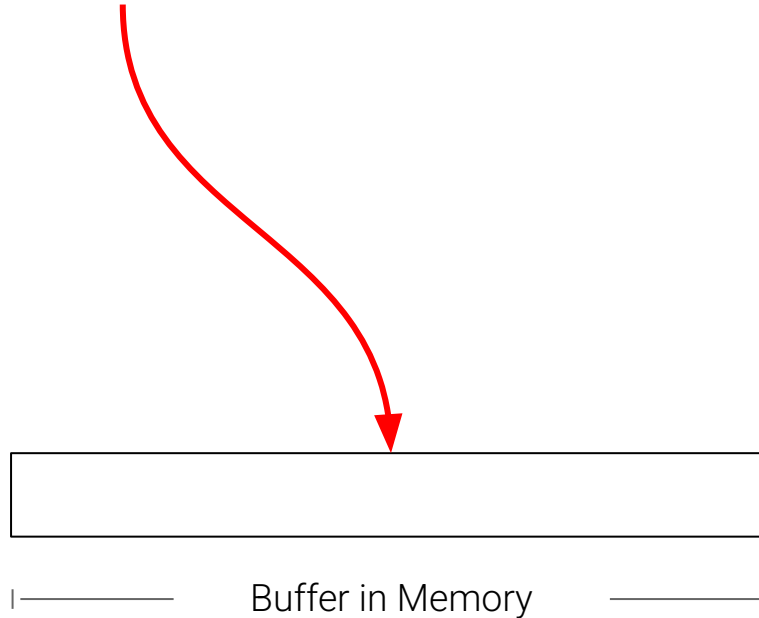
`char *_IO_buf_end`

`char *_IO_read_end`



FILE struct - Writing from Memory to File

When we call `fwrite(buf, 1, 10, fp)` It writes to a buffer, not the file!



FILE struct - Writing from Memory to File

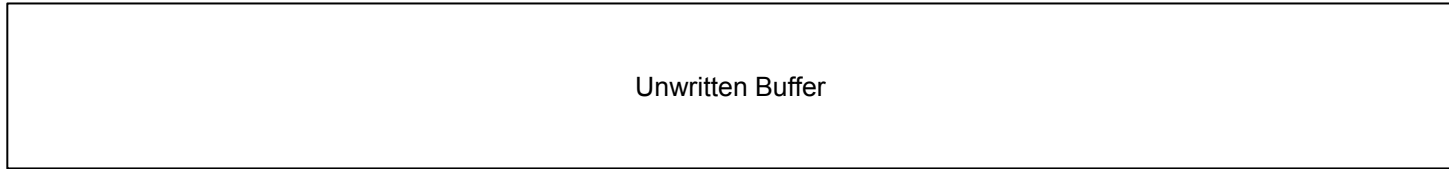
`char *_IO_write_ptr`

`char *_IO_write_base`

`char *_IO_buf_base`

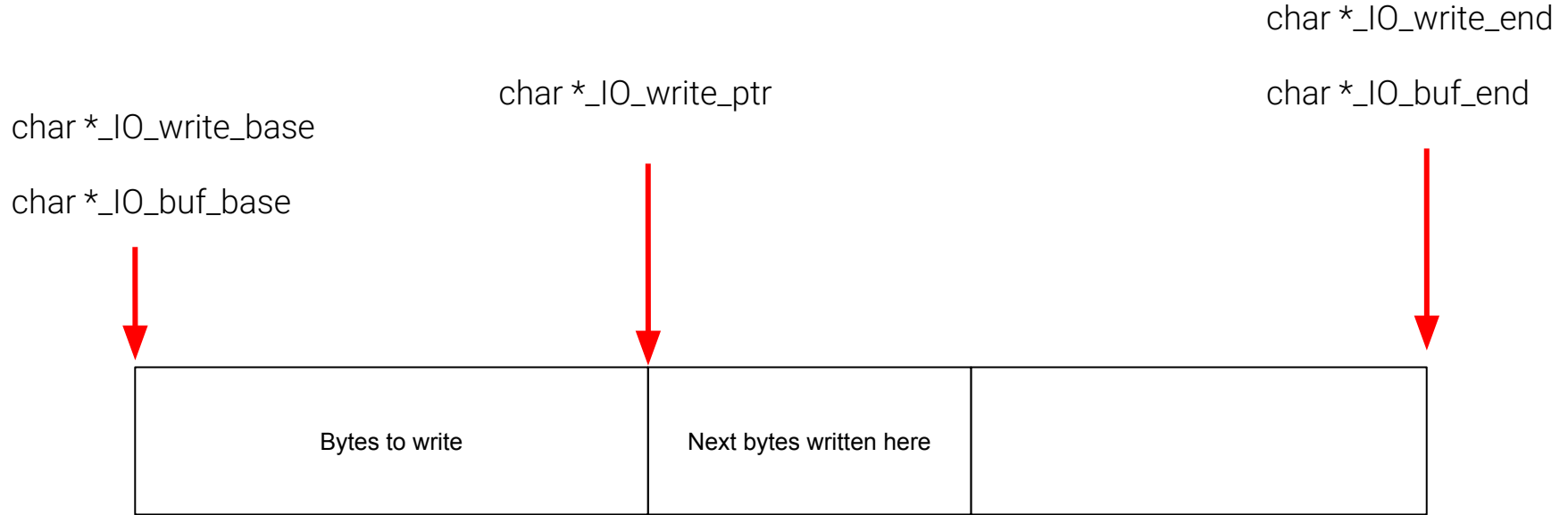
`char *_IO_write_end`

`char *_IO_buf_end`



*File must be opened for write!

FILE struct - Writing from Memory to File



*File must be opened for write!

FILE struct - Writing from Memory to File

char *_IO_write_base

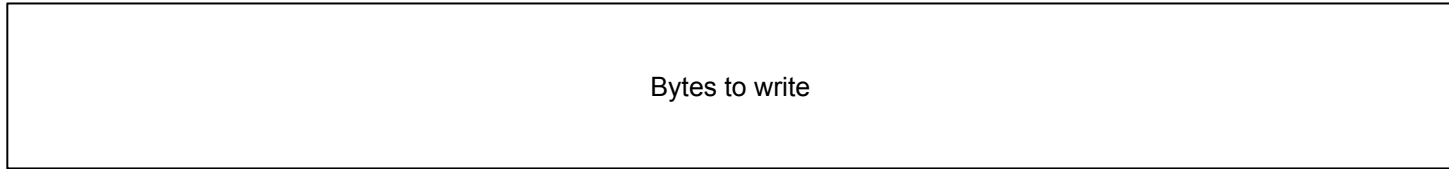
char *_IO_buf_base



char *_IO_write_ptr

char *_IO_write_end

char *_IO_buf_end



Bytes to write

*File must be opened for write!

FILE struct - Writing from Memory to File - Flushed

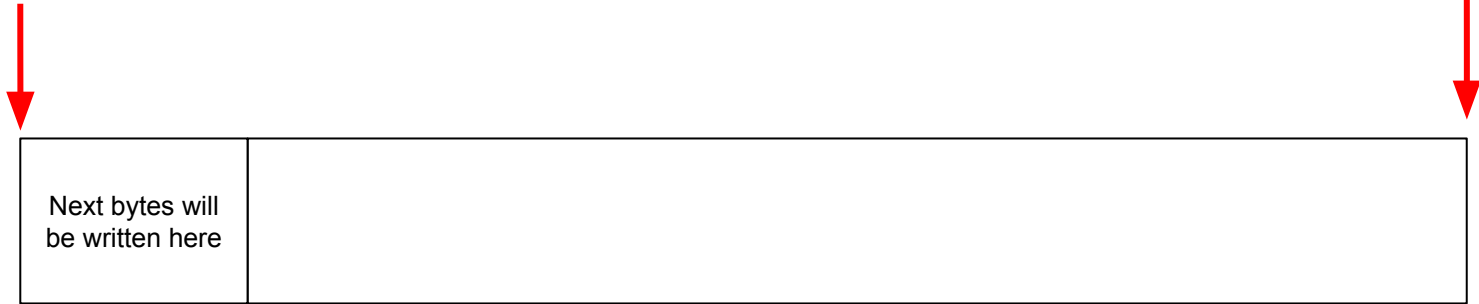
char *_IO_write_ptr

char *_IO_write_base

char *_IO_buf_base

char *_IO_write_end

char *_IO_buf_end



*File must be opened for write!

FILE struct

Where is this FILE struct located??

In Userland!